

NumPy — Nền tảng tính toán cho AI

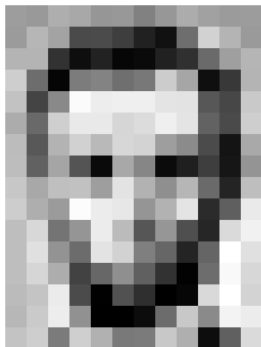
Thư viện cho Trí tuệ nhân tạo

Trình bày: Võ Luyện

Nội dung buổi học

- 1 Vì sao AI cần NumPy?
- 2 Bắt đầu với NumPy
- 3 Các cách tạo mảng
- 4 Thuộc tính và kiểu dữ liệu
- 5 Truy cập phần tử: indexing và slicing
- 6 View và Copy — cái bẫy kinh điển
- 7 Vector hóa — tính toán không cần vòng lặp
- 8 Broadcasting
- 9 Hàm thống kê và khái niệm axis
- 10 Biến đổi hình dạng và ghép mảng
- 11 Đại số tuyến tính với NumPy
- 12 NumPy trong các bài toán AI
- 13 Mẹo hay và bảng tra cứu
- 14 Thực hành

Dữ liệu trong AI là những con số — rất nhiều con số



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

- Máy tính **không "nhìn thấy" ảnh** — nó chỉ thấy một **ma trận các con số**, mỗi số là độ sáng của một điểm ảnh (pixel), từ 0 (đen) đến 255 (trắng)
- Chỉ một ảnh bé xíu như trên đã là hàng trăm con số. Một tấm ảnh 224×224 màu $\rightarrow 224 \times 224 \times 3 = 150\,528$ con số
- Bảng điểm 1000 học sinh $\times$ 10 môn $\rightarrow 10\,000$ con số

Vậy xử lý ngàn ấy con số bằng gì?

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số một, chuyện gì sẽ xảy ra?

Vậy xử lý ngàn ấy con số bằng gì?

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số một, chuyện gì sẽ xảy ra?

Trả lời

Chương trình chạy **rất chậm**. Với hàng triệu con số, thứ đáng lẽ chạy trong vài giây có thể mất hàng phút, thậm chí hàng giờ.

Vậy xử lý ngàn ấy con số bằng gì?

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số một, chuyện gì sẽ xảy ra?

Trả lời

Chương trình chạy **rất chậm**. Với hàng triệu con số, thứ đáng lẽ chạy trong vài giây có thể mất hàng phút, thậm chí hàng giờ.

Lời giải

Ta cần một công cụ chuyên để chứa và tính toán trên **cả một khối số cùng lúc** — đó chính là **NumPy**.

NumPy là gì?

- **NumPy** (Numerical Python) — thư viện tính toán số học nền tảng của Python
- Ra đời năm 2006, hiện là "trái tim" của toàn bộ hệ sinh thái khoa học dữ liệu và AI
- Cung cấp kiểu dữ liệu **ndarray** — mảng nhiều chiều, lưu số hiệu quả
- Phần lõi được viết bằng **C** và Fortran → nhanh hơn Python thuần hàng chục đến hàng trăm lần

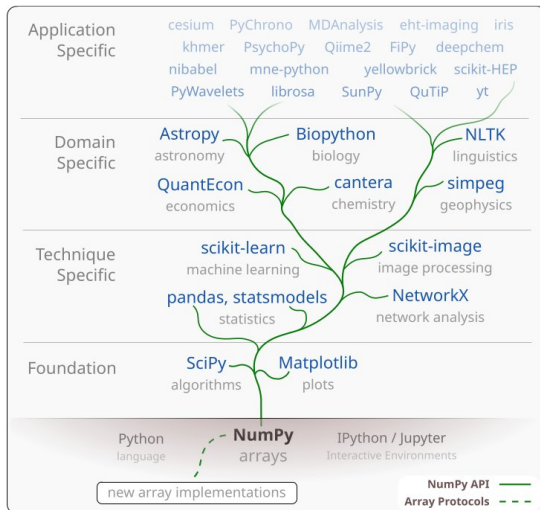
NumPy là gì?

- **NumPy** (Numerical Python) — thư viện tính toán số học nền tảng của Python
- Ra đời năm 2006, hiện là "trái tim" của toàn bộ hệ sinh thái khoa học dữ liệu và AI
- Cung cấp kiểu dữ liệu **ndarray** — mảng nhiều chiều, lưu số hiệu quả
- Phần lõi được viết bằng **C** và Fortran → nhanh hơn Python thuần hàng chục đến hàng trăm lần

Ai đang dùng NumPy?

Pandas, Scikit-learn, Matplotlib, OpenCV, PyTorch, TensorFlow — tất cả đều xây dựng trên NumPy hoặc dùng chung cách tổ chức dữ liệu của NumPy.

Vị trí của NumPy trong hệ sinh thái AI



Nguồn: Harris et al., *Array programming with NumPy*, Nature 585 (2020)

So sánh tốc độ: list vs ndarray

```
import numpy as np, time

n = 1_000_000
list_a = list(range(n))
arr_a = np.arange(n)

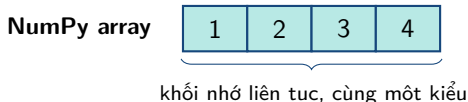
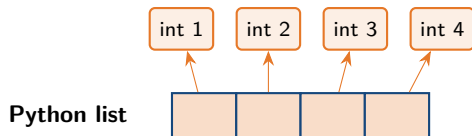
t0 = time.time()
tong1 = sum(x * x for x in list_a)    # Python thuan
t1 = time.time()
tong2 = np.sum(arr_a * arr_a)         # NumPy
t2 = time.time()

print("Python list:", t1 - t0, "giây")
print("NumPy      :", t2 - t1, "giây")
```

Kết quả tham khảo (tùy máy)

Python list $\approx$ 0.025–0.09 giây; NumPy $\approx$ 0.001–0.002 giây $\rightarrow$ **nhANH hơn 20–50 lần**

Vì sao NumPy nhanh hơn?



Mỗi ô chỉ là **con trỏ** tới một đối tượng Python nằm rải rác trong bộ nhớ. Mỗi phép cộng phải: tìm đối tượng → kiểm tra kiểu → tính → tạo đối tượng mới.

Các số nằm **liền nhau**, cùng kiểu → vòng lặp chạy trong C, tận dụng cache CPU và lệnh xử lý song song (SIMD).

NumPy array khác Python list ở điểm nào?

	Python list	NumPy ndarray
Kiểu phần tử	Trộn lẫn tùy ý	Đồng nhất (int, float, ...)
Kích thước	Thay đổi được	Cố định khi tạo
Bộ nhớ	Nhiều, rải rác	Ít, liên tục
Tốc độ tính toán	Chậm (vòng lặp)	Nhanh (chạy trong C)
Phép toán số học	$[1, 2] + [3, 4] \rightarrow$ nối	$a + b \rightarrow$ cộng từng phần tử
Nhiều chiều	Phải lồng list	Hỗ trợ sẵn
Hàm toán học	Phải tự viết	Có sẵn hàng trăm hàm

Ghi nhớ

List dùng để chứa **mọi thứ**. NumPy array dùng để **tính toán trên số**.

Cài đặt và import

Cài đặt (chỉ làm một lần, trong Terminal / Command Prompt):

```
pip install numpy
```

Với Google Colab hoặc Anaconda: NumPy đã được cài sẵn. **Import** — quy ước chung của cả thế giới:

```
import numpy as np

print(np.__version__)    # kiểm tra phiên bản
```

as np nghĩa là gì?

as đặt cho thư viện một **tên gọi tắt** (tiếng Anh: *alias*). Sau dòng đó, thay vì viết `numpy.array(...)` ta chỉ cần viết `np.array(...)` — ngắn hơn, gõ nhanh hơn. Cả thế giới đều dùng tên tắt `np`, nên hãy viết y hệt vậy để đọc code của người khác không bị lạ.

Mảng đầu tiên của bạn

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a)           # [1 2 3 4 5]
print(type(a))     # <class 'numpy.ndarray'>

# Phep toan tac dong len TOAN BO mang
print(a + 10)      # [11 12 13 14 15]
print(a * 2)       # [ 2  4  6  8 10]
print(a ** 2)      # [ 1  4  9 16 25]
```

- Không cần vòng lặp `for` — đây gọi là **vector hóa** (vectorization)
- So sánh: với list, `[1,2,3] * 2` cho `[1,2,3,1,2,3]` — hoàn toàn khác!

Mảng nhiều chiều — khái niệm cốt lõi

3	1	4	1
---	---	---	---

1 chiều (1D)

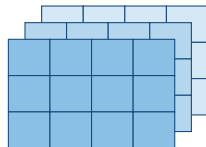
vector — shape (4,)

vd: điểm 4 môn học

2 chiều (2D)

ma trận — shape (3,4)

vd: bảng điểm 3 HS $\times$ 4 môn



3 chiều (3D)

tensor — shape (3,4,3)

vd: ảnh màu cao $\times$ rộng $\times$ RGB

Thuật ngữ: **tensor**

Trong AI, mảng nhiều chiều được gọi chung là **tensor**: một số lẻ là tensor 0 chiều, vector là 1 chiều, ma trận là 2 chiều, từ 3 chiều trở lên vẫn gọi là tensor. Mọi dữ liệu — ảnh, âm thanh, văn bản, bảng số — đều được biểu diễn bằng tensor.

Tạo mảng 2 chiều

```
diem = np.array([[8.0, 7.5, 9.0],      # hoc sinh 1
                 [6.5, 8.0, 7.0],      # hoc sinh 2
                 [9.0, 9.5, 8.5]])      # hoc sinh 3

print(diem)
print(diem.shape)    # (3, 3)  -> 3 hang, 3 cot
print(diem.ndim)     # 2       -> so chieu
```

- Mảng 2D được tạo từ **list của các list**
- Mỗi list con là một **hàng** (row) và phải có **cùng độ dài**
- Quy ước đọc shape: (số hàng, số cột)

Tạo mảng từ giá trị có sẵn

```
np.zeros(shape, dtype)    # CU PHAP: hình dạng, roi kieu du lieu

np.zeros(5)               # shape = 5          -> [0. 0. 0. 0. 0.]
np.zeros((2, 3))          # shape = (2, 3)     -> ma tran 2x3 toan so 0
np.ones((2, 3))           # ma tran 2x3 toan so 1
np.full((2, 3), 7)        # (shape, gia tri) -> 2x3 toan so 7
np.eye(3)                 # ma tran don vi 3x3 (duong cheo = 1)
```

- Tham số **thứ nhất** là **shape** — hình dạng. Nhiều chiều thì phải gói tất cả vào **một** cặp ngoặc: `(2, 3)`. Đó là lý do `np.zeros((2, 3))` có **hai** lớp ngoặc
- Tham số **thứ hai** là **dtype** — kiểu dữ liệu, **không phải** số cột

Vì vậy `np.zeros(2, 3)` sẽ báo lỗi

NumPy hiểu thành `shape = 2, dtype = 3` — mà 3 không phải là một kiểu dữ liệu.

Tạo dãy số: arange và linspace

```
np.arange(start, stop, step)    # đếm từ start, mỗi lần +step  
np.linspace(start, stop, num)  # chia đều đoạn [start, stop]  
  
np.arange(5)                    # [0 1 2 3 4]          stop = 5  
np.arange(2, 10)                # [2 3 4 5 6 7 8 9]    start=2, stop=10  
np.arange(0, 10, 2)             # [0 2 4 6 8]        step = 2  
np.linspace(0, 1, 5)            # [0. 0.25 0.5 0.75 1.] num = 5 so
```

Tham số	Ý nghĩa
start	giá trị bắt đầu — luôn được lấy
stop	giá trị dừng: arange không lấy, linspace có lấy
step	bước nhảy — chỉ arange có
num	số phần tử — chỉ linspace có

- Vẽ đồ thị hàm số → dùng **linspace**; tạo chỉ số đếm → dùng **arange**

Tạo mảng ngẫu nhiên

```
rng = np.random.default_rng(42)    # 42 là "hat giống" (seed)

rng.random(5)                       # 5 số thực ngẫu nhiên trong [0, 1)
rng.random((2, 3))                  # ma trận 2x3 số thực ngẫu nhiên
rng.integers(1, 7, 10)              # 10 số nguyên 1..6 (giao xúc xắc)
rng.normal(0, 1, 5)                 # 5 số phân phối chuẩn (mean 0, std 1)
```

Vì sao cần seed?

Đặt seed → lần chạy nào cũng cho **cùng một kết quả** → thí nghiệm AI có thể **lặp lại được** (reproducible). Đây là nguyên tắc bắt buộc khi làm nghiên cứu.

- Cách cũ (vẫn gặp nhiều trong tài liệu): `np.random.seed(42)` rồi `np.random.rand(5)`

Các thuộc tính quan trọng của mảng

```
a = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

Thuộc tính	Ý nghĩa	Với a ở trên
a.shape	hình dạng: (số hàng, số cột)	(2, 3)
a.ndim	số chiều	2
a.size	tổng số phần tử	6
a.dtype	kiểu dữ liệu của phần tử	int64
a.itemsize	số byte của mỗi phần tử	8
a.nbytes	tổng số byte (size × itemsize)	48

- Đây là **thuộc tính**, không phải hàm → viết `a.shape`, **không** có dấu ngoặc

Thói quen tốt số 1

Gặp lỗi hay nghi ngờ → `print(x.shape)` **ngay lập tức**. Phần lớn lỗi khi làm việc với mảng đến từ **sai hình dạng**.

Kiểu dữ liệu (dtype)

```
np.array([1, 2, 3]).dtype           # int64 (Windows cũ: int32)
np.array([1.0, 2.0]).dtype          # float64
np.array([1, 2], dtype=np.float32) # ép kiểu ngay khi tạo

a = np.array([1.7, 2.3, 3.9])
b = a.astype(int)                  # [1 2 3] - CAT phân thập phân
c = a.astype(np.float32)
```

Kiểu	Bộ nhớ	Dùng khi
uint8	1 byte	Ảnh (giá trị 0–255)
int32/int64	4/8 byte	Nhãn, chỉ số, số đếm
float32	4 byte	Số thực, tiết kiệm bộ nhớ
float64	8 byte	Mặc định của NumPy, độ chính xác cao
bool	1 byte	Mặt nạ lọc dữ liệu

Cẩn thận với kiểu số nguyên

```
a = np.array([250, 251, 252], dtype=np.uint8)
print(a + 10)          # [4 5 6] ??? -- TRAN SƠ (overflow)!
# uint8 chỉ chứa được 0..255, 260 quay vòng về 4

b = np.array([1, 2, 3])
b[0] = 3.9
print(b)               # [3 2 3] -- bị cắt thành số nguyên
```

Quy tắc an toàn khi làm AI

Trước khi tính toán trên ảnh, luôn đổi sang số thực: `img = img.astype(np.float32) / 255.0`

Indexing và slicing mảng 1 chiều

```
a = np.array([10, 20, 30, 40, 50, 60])
```

```
a[0]          # 10    phan tu dau tien  
a[-1]         # 60    phan tu cuoi cung  
a[1:4]        # [20 30 40]  tu chi so 1 den 3  
a[:3]         # [10 20 30]  3 phan tu dau  
a[3:]         # [40 50 60]  tu chi so 3 den het  
a[::2]        # [10 30 50]  cach 1 lay 1  
a[::-1]       # [60 50 40 30 20 10]  dao nguoc
```

- Cú pháp giống hệt list của Python: `a[start : stop : step]`
— bắt đầu, kết thúc, bước nhảy
- Luôn nhớ: chỉ số bắt đầu từ **0**, và **không** lấy phần tử ở vị trí `stop`

Indexing mảng 2 chiều

```
a = np.array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12]])

a[1, 2]      # 7      hang 1, cot 2
a[1][2]      # 7      cach viet cua list - cham, TRANH DUNG
a[0]         # [1 2 3 4]    toan bo hang 0
a[:, 1]      # [2 6 10]    toan bo cot 1
a[0:2, 1:3]  # [[2 3], [6 7]] mang con 2x2
a[-1, -1]   # 12         phan tu goc duoi phai
```

Cách đọc

Dấu phẩy ngăn cách các chiều: `a[i, j]` — hàng `i`, cột `j`. Dấu hai chấm : nghĩa là "lấy tất cả".

Minh họa slicing 2 chiều

1	2	3	4
5	6	7	8
9	10	11	12

`a[0]`

1	2	3	4
5	6	7	8
9	10	11	12

`a[:, 1]`

1	2	3	4
5	6	7	8
9	10	11	12

`a[0:2, 1:3]`

Lọc dữ liệu bằng điều kiện (boolean masking)

```
diem = np.array([8.5, 4.0, 7.0, 9.5, 5.5, 3.0])

mask = diem >= 5
print(mask)           # [True False True True True False]
print(diem[mask])     # [8.5 7.  9.5 5.5]   chỉ lấy diem >= 5
print(diem[diem >= 5]) # viết gọn, rất phổ biến

print(np.sum(diem >= 5)) # 4 - đếm số học sinh đạt
print(np.mean(diem >= 5)) # 0.666... - tỉ lệ đạt

# Kết hợp nhiều điều kiện: dùng & (và), | (hoặc), ~ (phủ định)
print(diem[(diem >= 5) & (diem < 9)]) # [8.5 7.  5.5]
```

Chú ý

Dùng **&**, **|** chứ **không** dùng **and**, **or**. Và **bắt buộc** có dấu ngoặc quanh mỗi điều kiện.

Fancy indexing và np.where

Lấy nhiều phần tử theo danh sách chỉ số:

```
a = np.array([10, 20, 30, 40, 50])
a[[0, 2, 4]]           # [10 30 50]
a[[4, 4, 0]]           # [50 50 10] - lặp lại được, thu tu tùy ý
```

np.where — "nếu ...thì ...ngược lại":

```
diem = np.array([8.5, 4.0, 7.0, 3.0])

np.where(diem >= 5, 1, 0)      # [1 0 1 0] - gan nhan Dat/Truot
np.where(diem >= 5, diem, 0)   # [8.5 0. 7. 0.] - truot -> 0
np.where(diem >= 5)            # (array([0, 2]),) - vi tri
```

- np.where chính là cách viết if-else cho cả mảng, không cần vòng lặp

Slicing tạo ra "view", không phải bản sao

```
a = np.array([1, 2, 3, 4, 5])  
b = a[1:4]          # b là VIEW - nhìn vào cùng vùng nhớ với a  
b[0] = 99  
  
print(b)            # [99  3  4]  
print(a)            # [ 1 99  3  4  5]  <-- a CÙNG BI ĐỔI!
```

Slicing tạo ra "view", không phải bản sao

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]          # b là VIEW - nhìn vào cùng vùng nhớ với a
b[0] = 99

print(b)            # [99  3  4]
print(a)            # [ 1 99  3  4  5]  <-- a CÙNG BI ĐOI!
```

Muốn có bản sao độc lập → dùng `.copy()`:

```
c = a[1:4].copy()
c[0] = 0
print(a)            # [ 1 99  3  4  5]  <-- a không đổi
```

Vì sao NumPy làm vậy?

Để **tiết kiệm bộ nhớ** — cắt một mảng 1GB mà phải sao chép thì rất tốn kém.

Khi nào là view, khi nào là copy?

Thao tác	Kết quả
<code>b = a[1:4]</code>	View — dùng chung bộ nhớ
<code>b = a[a > 5]</code>	Copy (boolean masking)
<code>b = a[[0, 2]]</code>	Copy (fancy indexing)
<code>b = a.reshape(2, 3)</code>	View (nếu có thể)
<code>b = a.T</code>	View
<code>b = a.copy()</code>	Copy
<code>b = a + 0</code>	Copy (mảng mới từ phép toán)

```
b = a[1:4]
print(b.base is a)    # True  -> b là view của a
```

Quy tắc vàng cho người mới

Nếu bạn định **sửa** dữ liệu và **không** muốn ảnh hưởng mảng gốc → luôn gọi `.copy()`.

Phép toán trên từng phần tử (element-wise)

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

a + b      # [11 22 33 44]
b - a      # [ 9 18 27 36]
a * b      # [ 10 40 90 160]  nhan TUNG PHAN TU
b / a      # [10. 10. 10. 10.]
a ** 2     # [ 1  4  9 16]
b % 3      # [1 2 0 1]

a > 2      # [False False  True  True]
```

- Hai mảng phải có **cùng hình dạng** (hoặc tuân theo quy tắc broadcasting ở phần sau)
- Kết quả luôn là một mảng **mới**, cùng hình dạng

Các hàm toán học có sẵn (universal functions)

```
a = np.array([1, 4, 9, 16])

np.sqrt(a)          # [1.  2.  3.  4.]      can bac hai
np.exp(a)           # e^a                ham mu
np.log(a)           # ln(a)              logarit tu nhien
np.abs(-a)          # gia tri tuyet doi
np.round(np.array([1.234, 5.678]), 1)    # [1.2 5.7]

x = np.linspace(0, np.pi, 5)
np.sin(x)           # ham luong giac
np.maximum(a, 5)     # [ 5  5  9 16]      so sanh tung phan tu voi 5
```

Vì sao quan trọng với AI?

Các hàm hay gặp trong AI — sigmoid, tanh, ReLU, softmax (sẽ gặp ở cuối buổi) — đều được viết chỉ bằng vài hàm trên.

Vector hóa: viết lại vòng lặp cho gọn và nhanh

Bài toán

Mảng `diem` chứa điểm của cả lớp. Cần quy đổi sang thang mới theo công thức **điểm mới** = **điểm cũ** $\times 2 + 1$ cho **mọi** học sinh.

Cách viết của người mới — duyệt từng học sinh một:

```
ket_qua = []                                # danh sách rỗng
for i in range(len(diem)):                  # duyệt từng học sinh
    ket_qua.append(diem[i] * 2 + 1)         # tính rồi thêm vào cuối
ket_qua = np.array(ket_qua)                # đổi lại thành mảng NumPy
```

Cách viết NumPy — làm một lần cho cả mảng:

```
ket_qua = diem * 2 + 1
```

- Cùng kết quả, 1 dòng thay cho 4 dòng, và nhanh hơn hàng chục lần

Broadcasting — khi hai mảng khác hình dạng

```
a = np.array([1, 2, 3])
print(a + 10)          # [11 12 13] - so 10 duoc "phat" ra ca mang

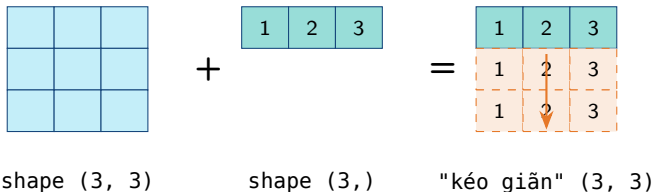
diem = np.array([[8.0, 7.0, 9.0],      # 3 hoc sinh
                 [6.0, 8.5, 7.5],      # x 3 mon
                 [9.0, 9.0, 8.0]])
he_so = np.array([2, 1, 1])           # Toan he so 2, Ly 1, Hoa 1

print(diem * he_so)                   # moi HANG deu duoc nhan voi he_so
```

Broadcasting là gì?

NumPy tự động "kéo giãn" mảng nhỏ hơn cho khớp hình dạng mảng lớn — **không tốn thêm bộ nhớ**, không cần vòng lặp.

Minh họa broadcasting



- Hàng [1 2 3] được lặp lại cho **mọi hàng** của ma trận
- Việc "kéo giãn" chỉ là **khái niệm** — NumPy không thực sự tạo bản sao trong bộ nhớ

Quy tắc broadcasting

So sánh hình dạng **từ phải sang trái**. Hai chiều tương thích khi:

- 1 Chúng **bằng nhau**, hoặc
- 2 Một trong hai **bằng 1**, hoặc
- 3 Một mảng **không có** chiều đó (được coi như bằng 1)

Hình dạng A	Hình dạng B	Kết quả
(3, 4)	(4,)	(3, 4) — OK
(3, 4)	(3, 1)	(3, 4) — OK
(3, 1)	(1, 4)	(3, 4) — OK, giãn cả hai
(3, 4)	(3,)	Lỗi — 4 và 3 không khớp

```
ValueError: operands could not be broadcast together  
with shapes (3,4) (3,)
```

Ứng dụng broadcasting: chuẩn hóa dữ liệu

Chuẩn hóa (normalization): đưa các cột dữ liệu về cùng một thang đo, để cột có giá trị lớn không lấn át cột có giá trị nhỏ.

```
# X: 100 mau du lieu, moi mau co 4 dac trung (features)
X = rng.normal(50, 10, size=(100, 4))

mean = X.mean(axis=0)    # shape (4,) - trung binh TUNG COT
std  = X.std(axis=0)     # shape (4,) - do lech chuan tung cot

X_chuan = (X - mean) / std    # broadcasting: (100,4) voi (4,)

print(X_chuan.mean(axis=0))  # ~ [0. 0. 0. 0.]
print(X_chuan.std(axis=0))   # ~ [1. 1. 1. 1.]
```

- Chỉ **một dòng** thay cho hai vòng lặp lồng nhau và 400 phép tính thủ công
- Đây gọi là chuẩn hóa **z-score** — công thức $z = (x - \mu) / \sigma$

Các hàm tổng hợp

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])
```

Hàm	Ý nghĩa	Với a ở trên
<code>a.sum()</code>	tổng các phần tử	31
<code>a.mean()</code>	trung bình cộng	3.875
<code>a.min()</code>	giá trị nhỏ nhất	1
<code>a.max()</code>	giá trị lớn nhất	9
<code>a.std()</code>	độ lệch chuẩn	2.571
<code>np.median(a)</code>	trung vị	3.5
<code>np.sort(a)</code>	sắp xếp tăng dần	[1 1 2 3 4 5 6 9]
<code>a.cumsum()</code>	tổng tích lũy	[3 4 8 9 14 23 25 31]

- Tất cả đều chạy trên **cả mảng**, không cần vòng lặp

axis — chiều nào bị "gộp lại"?

		axis=1			
axis=0		1	2	3	4
		5	6	7	8
		9	10	11	12

Lệnh	Ý nghĩa	Kết quả
<code>a.sum()</code>	tổng tất cả	78
<code>a.sum(axis=0)</code>	gộp các hàng → tổng mỗi cột	<code>[15 18 21 24]</code>
<code>a.sum(axis=1)</code>	gộp các cột → tổng mỗi hàng	<code>[10 26 42]</code>

Mẹo nhớ

`axis=k` nghĩa là "chiều thứ k **biến mất**". Mảng $(3, 4)$ với `axis=0` → còn lại $(4,)$.

Đố vui: chọn axis nào?

```
# 4 học sinh (hàng) x 3 môn: Toán, Lý, Hoa (cột)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)

Đố vui: chọn axis nào?

```
# 4 học sinh (hàng) x 3 môn: Toán, Lý, Hoa (cột)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` [8. 7.33 8.67 5.83]

Đố vui: chọn axis nào?

```
# 4 học sinh (hàng) x 3 môn: Toán, Lý, Hoa (cột)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)

Đố vui: chọn axis nào?

```
# 4 học sinh (hàng) x 3 môn: Toán, Lý, Hoa (cột)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`

Đố vui: chọn axis nào?

```
# 4 học sinh (hàng) x 3 môn: Toán, Lý, Hoa (cột)
diem = np.array([[8.0, 7.0, 9.0],
                  [6.0, 8.5, 7.5],
                  [9.0, 9.0, 8.0],
                  [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được

Đố vui: chọn axis nào?

```
# 4 học sinh (hàng) x 3 môn: Toán, Lý, Hoa (cột)
diem = np.array([[8.0, 7.0, 9.0],
                  [6.0, 8.5, 7.5],
                  [9.0, 9.0, 8.0],
                  [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được
→ `diem.max(axis=1)` `[9. 8.5 9. 6.5]`

Đố vui: chọn axis nào?

```
# 4 học sinh (hang) x 3 môn: Toán, Ly, Hoa (cot)
diem = np.array([[8.0, 7.0, 9.0],
                  [6.0, 8.5, 7.5],
                  [9.0, 9.0, 8.0],
                  [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được
→ `diem.max(axis=1)` `[9. 8.5 9. 6.5]`
- 4 Mỗi học sinh có **mấy môn** từ 8 điểm trở lên?

Đố vui: chọn axis nào?

```
# 4 học sinh (hang) x 3 môn: Toán, Ly, Hoa (cot)
diem = np.array([[8.0, 7.0, 9.0],
                  [6.0, 8.5, 7.5],
                  [9.0, 9.0, 8.0],
                  [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được
→ `diem.max(axis=1)` `[9. 8.5 9. 6.5]`
- 4 Mỗi học sinh có **mấy môn** từ 8 điểm trở lên?
→ `(diem >= 8).sum(axis=1)` `[2 1 3 0]`

Mẹo: giữ nguyên số chiều với keepdims

```
tb = diem.mean(axis=1)           # shape (4,)
tb = diem.mean(axis=1, keepdims=True) # shape (4, 1)

print(diem - tb)    # moi diem lech bao nhieu so voi DTB
                   # cua chinh hoc sinh do
```

- `keepdims=True` giữ lại chiều vừa bị gộp $\rightarrow$ shape $(4,1)$ thay vì $(4,)$
- Nhờ vậy `diem - tb` mới broadcasting đúng: mỗi **hàng** của `diem` trừ đi đúng giá trị trung bình của hàng đó

reshape — thay đổi hình dạng

```
a = np.arange(12)           # [0 1 2 ... 11]   shape (12,)
b = a.reshape(3, 4)         # 3 hàng, 4 cột
c = a.reshape(4, 3)         # 4 hàng, 3 cột
d = a.reshape(2, 2, 3)      # mang 3 chiều
e = a.reshape(3, -1)        # -1 = "tu tình giúp tôi" -> (3, 4)

print(b.ravel())             # [0 1 ... 11]   tra về 1 chiều (view)
print(b.flatten())           # giống ravel nhưng tra về BAN SẠO
```

- Số phần tử phải **không đổi**: $3 \times 4 = 12$ (được), còn $5 \times 3 = 15$ thì không
- -1 nghĩa là "chiều này để NumPy tự tính": `a.reshape(3, -1)`
→ NumPy suy ra 4 cột

Thêm chiều cho mảng

```
v = np.array([1, 2, 3])      # shape (3,) - không hàng, không cột  
  
v[:, np.newaxis]           # shape (3, 1) - vector CỘT  
v[np.newaxis, :]           # shape (1, 3) - vector HÀNG  
v.reshape(-1, 1)           # cách viết khác cho vector cột  
  
a = np.array([[1, 2, 3],  
               [4, 5, 6]])  # shape (2, 3)  
a.T                         # chuyển vị -> shape (3, 2) (xem phần sau)
```

Điểm dễ nhầm

Mảng shape (3,) khác shape (3,1) và khác shape (1,3). Rất nhiều lỗi broadcasting đến từ chỗ này — nhớ `print(x.shape)`!

Ghép và tách mảng

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.concatenate([a, b], axis=0)    # ghép theo HÀNG -> (4, 2)
np.concatenate([a, b], axis=1)    # ghép theo CỘT -> (2, 4)
np.vstack([a, b])                 # vertical = axis 0
np.hstack([a, b])                 # horizontal = axis 1
np.stack([a, b])                  # tạo chiều MỚI -> (2, 2, 2)

x = np.arange(10)
np.split(x, 2)                    # tách làm 2 phần bằng nhau
np.split(x, [3, 7])               # tách tại vị trí 3 và 7
```

- `np.stack` thêm một chiều mới — dùng khi gom nhiều ảnh cùng kích thước thành một chồng ảnh

Tích vô hướng của hai vector

Hai vector **cùng độ dài**: nhân từng cặp phần tử tương ứng rồi cộng tất cả lại.

$$\vec{u} \cdot \vec{v} = u_1 v_1 + u_2 v_2 + u_3 v_3 = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])

np.dot(u, v)           # 32
(u * v).sum()         # 32 - nhân từng cặp rồi cộng lại
np.linalg.norm(u)      # 3.74  độ dài của vector u
np.linalg.norm(u - v)  # 5.20  khoảng cách giữa u và v
```

- **norm** là **độ dài** vector — chính là định lý Pythagore mở rộng:
 $\sqrt{u_1^2 + u_2^2 + u_3^2} = \sqrt{1 + 4 + 9} \approx 3.74$
- **norm(u - v)** là **khoảng cách** giữa hai điểm — dùng để đo hai dữ liệu giống nhau đến đâu

Nhân ma trận được thực hiện như thế nào?

Quy tắc: phần tử ở **hàng** i , **cột** j của kết quả = tích vô hướng của **hàng** i (ma trận trái) với **cột** j (ma trận phải).

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \times \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

$$19 = 1 \times 5 + 2 \times 7$$

$$22 = 1 \times 6 + 2 \times 8$$

$$43 = 3 \times 5 + 4 \times 7$$

$$50 = 3 \times 6 + 4 \times 8$$

Điều kiện về hình dạng

$(m, n) @ (n, p) \rightarrow$ kết quả (m, p) . Số **cột** của ma trận trái phải bằng số **hàng** của ma trận phải — có vậy mới ghép cặp được từng phần tử.

Trong NumPy: * và @ là hai phép khác nhau

```
A = np.array([[1, 2],  
              [3, 4]])  
B = np.array([[5, 6],  
              [7, 8]])
```

```
A * B      # [[ 5 12], [21 32]]  nhân TUNG PHAN TU cùng vị trí  
A @ B      # [[19 22], [43 50]]  NHÂN MA TRẬN như trên  
np.dot(A, B)      # giống A @ B  
np.matmul(A, B)   # giống A @ B
```

Đừng nhầm!

* nhân từng phần tử cùng vị trí (đòi hỏi hai mảng cùng hình dạng). @ là phép nhân ma trận trong toán học. Hai kết quả hoàn toàn khác nhau.

Chuyển vị: đổi hàng thành cột

Chuyển vị (transpose) — lật ma trận qua đường chéo chính: hàng thứ i trở thành cột thứ i .

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}_{2 \times 3} \quad A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}_{3 \times 2}$$

- Phần tử ở vị trí (hàng i , cột j) chuyển sang vị trí (hàng j , cột i)
- Hình dạng (m, n) trở thành (n, m)
- Trong NumPy chỉ cần viết **A.T** — và đây là một **view**, không tốn thêm bộ nhớ
- Hay dùng khi hai mảng "lệch chiều" nhau, ví dụ muốn nhân $(2, 3)$ với $(2, 3)$: không nhân được, nhưng $A @ B.T$ thì được

Vì sao hệ phương trình viết được dưới dạng ma trận?

$$\begin{cases} 2x + y = 5 \\ x + 3y = 10 \end{cases} \iff \underbrace{\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}}_A \underbrace{\begin{pmatrix} x \\ y \end{pmatrix}}_{\vec{x}} = \underbrace{\begin{pmatrix} 5 \\ 10 \end{pmatrix}}_{\vec{b}}$$

Nhân **hàng 1** của A với vector ẩn theo đúng quy tắc ở slide trước:
 $2 \times x + 1 \times y$ — chính là vế trái của phương trình thứ nhất. Tương tự hàng 2 cho ra $x + 3y$.

```
A = np.array([[2, 1], [1, 3]])  
b = np.array([5, 10])  
  
np.linalg.solve(A, b)      # [1. 3.] -> x = 1, y = 3  
A @ np.array([1, 3])      # [ 5 10] kiểm tra lại: đúng!
```


Ảnh số chính là một mảng NumPy

```
# Ảnh mau: (cao, rong, 3 kênh RGB), kieu uint8 gia tri 0..255
img = rng.integers(0, 256, size=(128, 128, 3), dtype=np.uint8)

img.shape          # (128, 128, 3)
img[0, 0]          # [R G B] của diem anh goc tren trai

img_gray = img.mean(axis=2)          # anh xam: trung binh 3 kênh
img_flip = img[:, ::-1, :]           # lat anh trai-phai
img_crop = img[32:96, 32:96]         # cat vung giua
img_norm = img.astype(np.float32) / 255.0 # dua ve [0, 1]

do_sang = img.mean()                 # do sang trung binh
kênh_do = img[:, :, 0]               # tach riêng kênh do
```

- Lật ảnh, cắt ảnh, đổi độ sáng — các thao tác xử lý ảnh quen thuộc — chỉ là slicing và phép toán trên mảng

Vài hàm số hay gặp khi học AI

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

$$\text{ReLU}(x) = \max(0, x)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))  
  
def relu(x):  
    return np.maximum(0, x)  
  
x = np.linspace(-5, 5, 11)  
print(np.round(sigmoid(x), 2)) # [0.01 0.02 ... 0.98 0.99]  
print(relu(x))                 # [0. 0. ... 4. 5.]
```

- sigmoid nén **mọi** số thực về khoảng (0,1)
- ReLU giữ nguyên số dương, cắt số âm về 0
- Viết được cho **cả mảng** chỉ nhờ vector hóa — không cần vòng lặp

softmax — biến dãy số thành tỉ lệ phần trăm

$$\text{softmax}(x)_i = \frac{e^{x_i}}{e^{x_1} + e^{x_2} + \dots + e^{x_n}}$$

```
def softmax(x):  
    e = np.exp(x - np.max(x))    # tru max de tranh tran so  
    return e / np.sum(e)  
  
diem_so = np.array([2.0, 1.0, 0.1])  
print(softmax(diem_so))          # [0.659 0.242 0.099] - tong = 1
```

- Mọi kết quả đều **dương** và **tổng bằng 1** → đọc được như tỉ lệ phần trăm: 65.9%, 24.2%, 9.9%
- Số nào lớn hơn thì tỉ lệ cao hơn — nhưng chênh lệch bị khuếch đại (do hàm e^x)

Đo sai số giữa hai dãy số

Ví dụ: nhiệt độ *dự báo* và nhiệt độ *thực tế* của 4 ngày — dự báo lệch trung bình bao nhiêu độ?

```
du_bao    = np.array([28.0, 31.0, 27.5, 33.0])
thuc_te    = np.array([27.8, 31.2, 29.0, 32.5])

sai_so = du_bao - thuc_te          # [0.2 -0.2 -1.5 0.5]

# Sai số tuyệt đối trung bình (MAE)
np.mean(np.abs(sai_so))            # 0.6 độ

# Sai số bình phương trung bình (MSE) - phạt nặng sai số lớn
np.mean(sai_so ** 2)               # 0.645
```

Đếm tỉ lệ giống nhau giữa hai dãy:

```
dap_an    = np.array([1, 3, 2, 4, 1])    # đáp án đúng
bai_lam    = np.array([1, 3, 4, 4, 2])    # bài làm của học sinh
np.mean(dap_an == bai_lam)               # 0.6 -> làm đúng 60%
```

Mẹo giúp code nhanh và sạch

- Luôn in shape khi debug: `print(x.shape, x.dtype)`
- Tránh vòng lặp trên phần tử — tìm cách viết bằng phép toán mảng
- Đừng nối mảng trong vòng lặp — mỗi lần `np.append` là một lần sao chép toàn bộ:

```
# CHAM
kq = np.array([])
for i in range(10000):
    kq = np.append(kq, i * 2)

# NHANH: gom vào list rồi chuyển một lần
kq = np.array([i * 2 for i in range(10000)])

# NHANH NHAT: vector hóa hoàn toàn
kq = np.arange(10000) * 2
```

- Dùng `float32` thay `float64` khi dữ liệu rất lớn → tiết kiệm một nửa bộ nhớ

Bảng tra cứu nhanh

Việc cần làm	Lệnh
Tạo mảng từ list	<code>np.array([1,2,3])</code>
Mảng số 0 / số 1	<code>np.zeros((m,n)), np.ones((m,n))</code>
Dãy số	<code>np.arange(start,stop,step)</code> <code>np.linspace(start,stop,num)</code>
Số ngẫu nhiên	<code>rng = np.random.default_rng(42)</code>
Xem hình dạng	<code>a.shape, a.ndim, a.dtype</code>
Đổi kiểu	<code>a.astype(np.float32)</code>
Lấy hàng / cột	<code>a[i], a[:, j]</code>
Lọc theo điều kiện	<code>a[a > 5]</code>
if-else cho mảng	<code>np.where(cond, x, y)</code>
Sao chép thật sự	<code>a.copy()</code>
Thống kê	<code>a.sum(), a.mean(), a.std()</code>
Theo hàng / cột	<code>a.sum(axis=1), a.sum(axis=0)</code>
Đổi hình dạng	<code>a.reshape(m, -1)</code>
Chuyển vị	<code>a.T</code>
Ghép mảng	<code>np.vstack, np.hstack, np.concatenate</code>
Nhân ma trận	<code>A @ B</code>
Tích vô hướng	<code>np.dot(u, v)</code>
Khoảng cách / độ dài	<code>np.linalg.norm(a - b)</code>
Giải hệ phương trình	<code>np.linalg.solve(A, b)</code>
Sắp xếp	<code>np.sort(a), np.argsort(a)</code>

Bản đầy đủ hơn (tiếng Anh): [geeksforgeeks.org/python/numpy-cheat-sheet](https://www.geeksforgeeks.org/python/numpy-cheat-sheet)

Notebook thực hành

File: `numpy_practice.ipynb`

Notebook đi kèm buổi học: 12 phần thực hành bám sát thứ tự bài giảng, 10 bài tập tự làm và lời giải gợi ý ở cuối.

- **Cách 1 — Google Colab** (khuyến nghị, không cần cài gì):
 - Vào `colab.research.google.com` → Tập → Tải notebook lên
- **Cách 2 — máy cá nhân:**

```
pip install numpy jupyter
jupyter notebook numpy_practice.ipynb
```

- **Cách 3 — VS Code:** mở thẳng file `.ipynb` (cần extension Python + Jupyter)

Quan trọng

Hãy **tự gõ lại** từng dòng code thay vì copy-paste. Gõ tay giúp nhớ cú pháp nhanh hơn nhiều lần.

- 1 Tạo mảng các số chẵn từ 0 đến 20, in ra tổng và trung bình cộng
- 2 Tạo ma trận 3×3 chứa các số từ 1 đến 9, rồi in ra: hàng thứ hai, cột cuối cùng, và đường chéo chính
- 3 Cho mảng điểm của 10 học sinh, đếm số học sinh đạt (điểm ≥ 5) và in ra danh sách điểm của những em này
- 4 Tạo ma trận điểm 5 học sinh $\times$ 3 môn. Tính điểm trung bình mỗi học sinh, mỗi môn, và tìm học sinh có điểm trung bình cao nhất
- 5 Chuẩn hóa một mảng số bất kỳ về khoảng $[0, 1]$ theo công thức

$$x' = \frac{x - \min}{\max - \min}$$

Bài tập về nhà

- 1 Viết hàm `sigmoid(x)` và `relu(x)`, áp dụng lên mảng `np.linspace(-5, 5, 11)`
- 2 Cho hai mảng `du_bao` và `thuc_te`, viết hàm tính sai số MAE và MSE (không dùng vòng lặp)
- 3 Tạo ma trận ngẫu nhiên 100×5 , chuẩn hóa z-score theo từng cột, kiểm tra lại trung bình ≈ 0 và độ lệch chuẩn ≈ 1
- 4 Tạo "ảnh" ngẫu nhiên kích thước $(64, 64, 3)$ kiểu `uint8`. Chuyển sang ảnh xám, lật ngang, cắt vùng giữa 32×32
- 5 Cho hai ma trận 3×2 và 2×4 , nhân chúng bằng `@`. Tự nhân tay một phần tử bất kỳ để kiểm tra kết quả
- 6 Viết hệ phương trình
$$\begin{cases} 3x + 2y = 12 \\ x - y = 1 \end{cases}$$
 dưới dạng ma trận rồi giải bằng `np.linalg.solve`
- 7 **Nâng cao:** viết hàm `softmax(x)` và kiểm tra tổng các phần tử kết quả luôn bằng 1

Tóm tắt buổi học

- **ndarray** — mảng nhiều chiều, cùng kiểu, nằm liên tục trong bộ nhớ → nhanh
- **Vector hóa** — tính toán trên cả mảng, không cần vòng lặp
- **Broadcasting** — tự động khớp hình dạng giữa các mảng khác kích thước
- **shape và axis** — hai khái niệm phải nắm thật chắc; luôn `print(x.shape)`
- **View vs copy** — slicing dùng chung bộ nhớ; muốn độc lập thì `.copy()`
- **@** — nhân ma trận: hàng nhân cột rồi cộng lại

Buổi sau

Pandas — xử lý dữ liệu dạng bảng, và **Matplotlib** — trực quan hóa dữ liệu.

Tài liệu: numpy.org/doc/stable/user/absolute_beginners.html
Cheat sheet: [geeksforgeeks.org/python/numpy-cheat-sheet](https://www.geeksforgeeks.org/python/numpy-cheat-sheet)